Software Requirements Specification

COMP3211 Software Engineering | Fall 2026

Software Requirements Specification

Status: Final submission baseline Version: 1.0 System name: Hong Kong Student Second-hand PIM

1 Introduction

1.1 Purpose

This document defines the requirements for a command-line Personal Information Management (PIM) system that helps one Hong Kong university student manage information related to second-hand trades. It is the requirements baseline for the design, implementation, and testing stages of the COMP3211 project.

1.2 Scope

The system stores and manages Personal Information Records (PIRs). A typical trade is represented by a plain-text listing, one or more contacts, transaction tasks, and handover events. The system supports creating, modifying, searching, printing, deleting, saving, and loading those records.

The system is local and single-user. Payment, delivery, account registration, network communication, real-time chat, and a graphical interface are outside the graded scope.

1.3 Definitions

Term	Meaning
PIR	Personal Information Record managed by the PIM
Plain Text PIR	A free-form text record used for a listing description or note
Task PIR	A record with a description and deadline
Event PIR	A record with a description, starting time, and alarm
Contact PIR	A record with a name, address, and mobile number
Repository	The in-memory collection of PIRs in the current session
Criterion	A logical expression used to search PIRs
`.pim` file	The local file format used to save and load the repository

2 Overall description

2.1 User

The system has one user in each local session. The user may act as a seller, buyer, or both, and uses the PIM to manage the records for their own trades.

2.2 Product perspective

The product is a standalone command-line program. It follows an MVC structure:

- Model: PIR types, validation, search criteria, repository, and persistence.
- View: prompts, command results, detailed record output, and error messages.
- Controller: command parsing and coordination of model operations.

2.3 Operating constraints

- The implementation shall use Python and Python standard libraries only.
- Model code shall be placed in a separate package named `model`.

- The interaction shall be command-line based.
- Unit tests shall be automatically executable and shall target the system model.
- Required documents and reports shall be prepared for the final ZIP submission.

2.4 Assumptions

- A PIR receives a unique integer identifier when it is created.
- The user supplies date and time values in the format `YYYY-MM-DD HH:MM` using Hong Kong local time.
- A listing is represented by a Plain Text PIR. Its content shall use a documented template containing title, category, condition, price, availability, and description.
- Loading a `.pim` file replaces the current in-memory repository after the file is successfully validated.
- The exact serialization schema is finalized in the design stage, but the file extension is always `.pim`.

3 Functional requirements

The word **shall** identifies a mandatory requirement.

3.1 Record creation

FR-01 Create PIRs (US1). The system shall allow the user to create a PIR of each supported type: Plain Text, Task, Event, and Contact. Each new record shall receive a unique identifier and shall be added to the current repository.

FR-02 Create Plain Text PIRs (US2). The system shall allow the user to create a Plain Text PIR containing arbitrary text. The user manual shall explain the recommended listing template for second-hand items.

FR-03 Create Task PIRs (US3). The system shall allow the user to create a Task PIR with a non-empty description and a deadline.

FR-04 Create Event PIRs (US4). The system shall allow the user to create an Event PIR with a non-empty description, a starting time, and an alarm time.

FR-05 Create Contact PIRs (US5). The system shall allow the user to create a Contact PIR with a non-empty name, address, and mobile number.

3.2 Record maintenance

FR-06 Modify PIRs (US6). The system shall allow the user to modify the fields of an existing PIR without changing its identifier or type. The system shall reject fields that do not belong to the selected PIR type.

FR-07 Validate input. The system shall reject missing required fields, invalid date and time values, invalid PIR identifiers, unsupported commands, invalid search fields, and malformed `.pim` files. An invalid operation shall display a human-readable error and shall leave the repository unchanged.

3.3 Search and output

FR-08 Search PIRs (US7). The system shall allow the user to search PIRs using a criterion concerning record type, text fields, time fields, or a logical combination of other criteria. The required criterion language is specified in command-specification.md.

FR-09 Print PIRs (US8). The system shall print detailed information for one selected PIR or for all PIRs. Printing all PIRs shall use a stable order based on their identifiers.

FR-10 Delete PIRs (US9). The system shall allow the user to delete a selected PIR by identifier. Deleting a missing identifier shall produce an error and shall not change the repository.

3.4 Persistence

FR-11 Save PIRs (US10). The system shall save all current PIRs to a user-specified file whose extension is `.pim`. A successful save shall preserve every record type, identifier, and field value.

FR-12 Load PIRs (US11). The system shall load PIRs from a valid `.pim` file and make them available for subsequent search, print, modify, delete, and save operations. A failed load shall leave the current repository unchanged.

FR-13 Session commands. The system shall provide commands for help and exit. The help output shall describe supported commands, arguments, valid formats, and error behavior.

4 Information requirements

4.1 Common PIR data

Every PIR shall contain:

- `id`: unique positive integer within the repository;
- `type`: one of `text`, `task`, `event`, or `contact`.

4.2 Type-specific data

PIR type	Required fields	Second-hand use
Plain Text	`content`	Listing title, category, condition, price, availability description
Task	`description`, `deadline`	Prepare item, confirm payment, remove listing
Event	`description`, `start`, `alarm`	Inspection or campus handover
Contact	`name`, `address`, `mobile`	Buyer or seller contact

4.3 Listing content template

The requirements stage defines the following recommended content shape for a listing Plain Text PIR:

```
title: iPad Air 5
category: electronics
condition: good
price: HKD 2000
status: available
description: Includes case; campus handover preferred.
```

The system stores this as text. It does not need a separate Listing PIR type for the required user stories.

5 Search requirements

The criterion language shall support the following predicates:

- record type: `type = text`, `type = task`, `type = event`, or `type = contact`;
- text containment: `content contains "iPad"`, `description contains "payment"`, `name contains "Chan"`, `address contains "Hung Hom"`, or `mobile contains "9123"`;
- time comparison: `deadline < "2026-11-20 20:00"`, `start > "2026-11-01 09:00"`, or `alarm = "2026-11-01 08:30"`;
- logical negation: `! criterion`;
- logical conjunction: `criterion && criterion`;
- logical disjunction: `criterion || criterion`;
- grouping with parentheses.

Text `contains` comparisons shall be case-insensitive while preserving the original stored text.

The precedence shall be `!`, then `&&`, then `||`. A field that does not exist on a record shall make that record fail the predicate. An unknown field, invalid operator, or malformed date shall be reported as an invalid criterion.

6 External interface requirements

The program shall present a prompt, accept one command per input line, and return either a result or an error message. The proposed command syntax is documented in command-specification.md and is part of this Stage 1 baseline.

7 Non-functional requirements

NFR-01 Standard library. The implementation shall use only Python standard-library modules.

NFR-02 Model separation. The model shall be importable and testable without starting the command-line loop or writing terminal prompts.

NFR-03 Reliability. An invalid command, criterion, record, or file shall not terminate the program with an unhandled exception during a normal session.

NFR-04 Data integrity. A successful save followed by a load shall reproduce an equivalent repository, including identifiers, types, and field values.

NFR-05 Usability. The program shall provide help text, identify invalid input, state the expected format, and use consistent result and error prefixes.

NFR-06 Determinism. Printing and search results shall use a stable order so demonstrations and tests are reproducible.

NFR-07 Performance. On the documented development platform, normal commands over 1,000 PIRs shall complete within one second, excluding user typing and file-system delays outside the program's control.

NFR-08 Testability. Every public model operation shall have unit-test coverage for its successful path and its relevant validation failures.

NFR-09 Privacy. The program shall not transmit record content to a network service. The user is responsible for protecting local `.pim` files.

8 Out of scope

The graded system will not include a web front end, network server, multi-user accounts, real-time chat, payment processing, delivery tracking, recommendation algorithms, image hosting, or external APIs.

9 Requirements baseline

The requirements in this document, the acceptance criteria, and the command specification form the Stage 1 baseline. Changes after acceptance shall be recorded in the stage delivery record and reflected in the traceability matrix.